Tashfeen, Ahmad
CSCI 6213: Data Science Fundamentals

Homework 6

You're *not allowed* to use any libraries other than Numpy or Matplotlib.

Let $x_1, \ldots, x_n$ be a set of instances in $\mathbb{R}^p$ just like before and $c_1, \ldots, c_K$ be a set of K cluster centroids. Note that c_i are also in $\mathbb{R}^p$. Let $\mathbb{I}_{ij} \in \{0, 1\}$ be a variable such that $\mathbb{I}_{ij} = 1$ if x_i is assigned to cluster centered at c_j and $\mathbb{I}_{ij} = 0$ otherwise. In layman's terms, $\mathbb{I}_{ij}$ indicates whether x_i is in the jth cluster. The K–Means Clustering algorithm minimises the following objective function,

$$\sum_{i=1}^{n} \sum_{j=1}^{K} \mathbb{I}_{ij} [(x_i - c_j)^T (x_i - c_j)]$$

This objective function is essentially a measure of how far are the members of each cluster from their respective centroids or means. The K–Means Clustering algorithm is given bellow,

1) Initialise centroids or means c_j for $1 \leq j \leq K$ randomly in $\mathbb{R}^p$ or,

$$c_j \leftarrow x_j$$

2) Assign all x_i to the closest centroid c_u. I. e., for some u and all j,

$$\textbf{if} \quad [(x_i - c_u)^T (x_i - c_u)] < [(x_i - c_j)^T (x_i - c_j)] \quad \textbf{then} \quad \mathbb{I}_{iu} \leftarrow 1$$

3) Adjust the centroids or means according the new assignment in the last step,

$$c_j \leftarrow \sum_{i=1}^{n} \mathbb{I}_{ij} \left(\frac{x_i}{n} \right)$$

   Note that this is just assigning c_j to the new mean of the jth cluster.

4) Repeat from (2) till c_j stop changing or the change is less than some ε.

You may also find this algorithm (Algorithm 12.2) on page 523 of your textbook (Hatie et al.).

```python
import numpy as np
from matplotlib import pyplot as plt

# Don't change the seed or use a different Generator
RNG = np.random.default_rng(3)
normal = RNG.multivariate_normal

t, n = 3, 4*2500
X = normal([-t, t], np.identity(2), n // 4)
X = np.vstack((X, normal([t, t], np.identity(2), n // 4)))
X = np.vstack((X, normal([t, -t], np.identity(2), n // 4)))
X = np.vstack((X, normal([-t, -t], np.identity(2), n // 4)))
plt.title('Unlabelled Data Clusters')
plt.xlabel('x--axis')
plt.ylabel('y--axis')
plt.scatter(X[:, 0], X[:, 1], marker='.', color='k', alpha=0.3)
plt.show()
```

Listing 1. Python code to generate synthetic two dimensional data clusters. Download this file.

Question 1. Listing 1 plots the clusters shown in figure 1. Give your best guesses for the optimal centroids $c_j \in \mathbb{R}^2$ and $K = 4$.

Question 2. Implement the K–Means Clustering algorithm for the data in listing 1. Run the algorithm for $K \in \{1, 2, 3, 4\}$ and $\varepsilon = 10^{-5}$. Give your final converged centroids plotted over the clustered data. Plot each cluster with a distinct colour[1]. At the end you should have four converged (with different coloured clusters) plots for each value of K.

Question 3. Download the file `t10k-images-idx3-byte`. This is the binary file containing 1000 images of the test set from the the MNIST database of handwritten digits. You can refer to their website for more information about the data and its organisation. However, it is read and plotted for your inspection and further usage to implement the K–Means Clustering algorithm in listing 2.

```python
import numpy as np
from matplotlib import pyplot as plt

j = [0]
PATH = 't10k-images-idx3-ubyte'
RNG = np.random.default_rng(3)

with open(PATH, 'rb') as f:
    arr = bytearray(f.read())
    magic_number = int.from_bytes(arr[:4], byteorder='big')
    num_examples = int.from_bytes(arr[4:8], byteorder='big')
    rows = int.from_bytes(arr[8:12], byteorder='big')
    cols = int.from_bytes(arr[12:16], byteorder='big')
    data = np.array(arr[16:]).reshape(num_examples, rows, cols)


def inc(_=None):
    j[0] = (j[0] + 1) % data.shape[0]
    plt.cla()
    plt.title('$' + str(j[0]) + '^{th}$ Data Point')
    plt.imshow(data[j[0]], cmap='gray')
    plt.draw()


plt.gcf().canvas.mpl_connect('key_press_event', inc)
inc()
plt.show()

ep, K = 3.5, 10
X = data.reshape(data.shape[0], data.shape[1] * data.shape[2])

means = means = X[RNG.integers(0, X.shape[0], K)]  # initial means
# implement K-means here

rows, columns = 2, 5
fig = plt.figure(figsize=(15, 7))
for i, cj in enumerate(means):
    fig.add_subplot(rows, columns, i+1)
    plt.imshow(cj.reshape(28, 28), cmap='grey')

plt.tight_layout()
plt.show()
```

Listing 2. Python code that reads in the binary MNSIT test set data and plots a few data points. Pressing the space-bar changes the plot upon running the file. Download this file.

[1] use different shades of grey if you have difficulty seeing colour

1) Why do you think $K = 10$ maybe a good choice for this data?
2) What are the values of n and p?
3) What does each of the total p features represent in this data?
4) For $\varepsilon = 3.5$, finish implementing the K–Means Clustering algorithm after the line indicated by the comment in listing 2. The code in listing 2 should then plot your 10 approximately converged cluster centroids as 28 by 28 pixel images comprising of 2 rows and 5 columns. Give and discuss this plot. Why do you think the centroids c_j look the way they do?

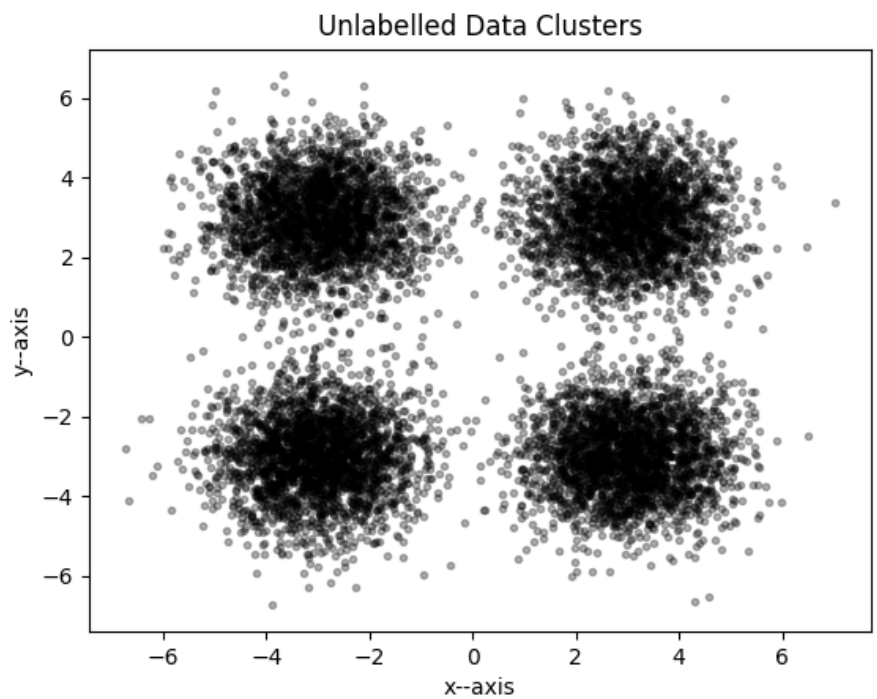

FIGURE 1. Unlabelled clusters of data.

SUBMISSION INSTRUCTIONS

1) Submit a PDF that answers the questions and contains all the plots that the assignment asks for.
2) Submit your `kmeans.py`. This is your K–Means implementation for the code in listing 1.
3) Submit your `mnsit.py`. This is your K–Means implementation for the MNSIT test set in reference to the listing 2.